ORCA ADE V1.0+ MODEL CONTEXT PROTOCOL (MCP)

LINEAR & GITHUB NATIVE ANTIGRAVITY AI AGENT

Orquestração de Agentes com Orca, Linear & MCP

Guia completo passo a passo: desde a instalação e login até a integração de múltiplos agentes de IA paralelos usando Git Worktrees isolados e ferramentas MCP nativas.

MÓDULO 01

Visão Geral da Arquitetura

Em um fluxo de engenharia de alta performance, o desenvolvedor não deve perder tempo alternando manualmente entre Jira/Linear, navegadores, terminais e branches do Git. O **Orca** atua como a central de comando unificada.



Orca (ADE)

O ambiente de desenvolvimento de agentes. Gerencia Git Worktrees isolados, terminais WebGL, preview de tela e diffs lado a lado.



Linear

A fonte da verdade para tarefas, requisitos, bugs e ciclos de entrega rápida com tracking ágil.



GitHub

Repositório de código onde branches de worktree são sincronizadas, testes rodam e Pull Requests são avaliados.



Camada MCP

Permite que o Agente de IA consulte tickets, comente no Linear e abra PRs no GitHub sem intervenção humana.

💡 Por que essa combinação é revolucionária?

Você seleciona um card no Linear, o Orca cria instantaneamente um **Git Worktree** isolado em background. O agente de IA codifica, roda os testes e abre o Pull Request no GitHub com rastreabilidade completa. Zero conflito de branches locais!

MÓDULO 02

Instalação e Login nos Serviços

1

Instalação e Primeiro Login no Orca

O Orca está disponível para Windows, macOS e Linux diretamente pelo repositório open-source `stablyai/orca`.

Comandos de Instalação por Sistema

```
# macOS (Homebrew)
brew install --cask stablyai/orca/orca

# Windows
# Baixe o instalador executável 'Orca-Setup.exe' em:
# https://github.com/stablyai/orca/releases

# Linux (Arch AUR)
yay -S stably-orca-bin
```

◆ Ao abrir o aplicativo pela primeira vez, conclua o assistente de boas-vindas e faça login com sua conta (ou configure provedores de IA preferidos em Settings).

2

Configurar Conta e Gerar API Key no Linear

Para permitir que tanto o Orca quanto o Agente de IA interajam com suas tarefas, gere um Personal API Key.

1. Acesse linear.app e faça login no seu Workspace.
2. No canto superior esquerdo, clique no nome do seu Workspace → **Settings**.
3. No menu lateral sob **Account**, clique em **Security & Access**.
4. Role até a seção **Personal API Keys** e clique em **New API Key**.
5. Defina o rótulo como `Orca-MCP-Agent` e clique em **Create**.
6. Copie a chave (formato: `lin_api_...`) e guarde em um local seguro.

3

Gerar Personal Access Token (PAT) no GitHub

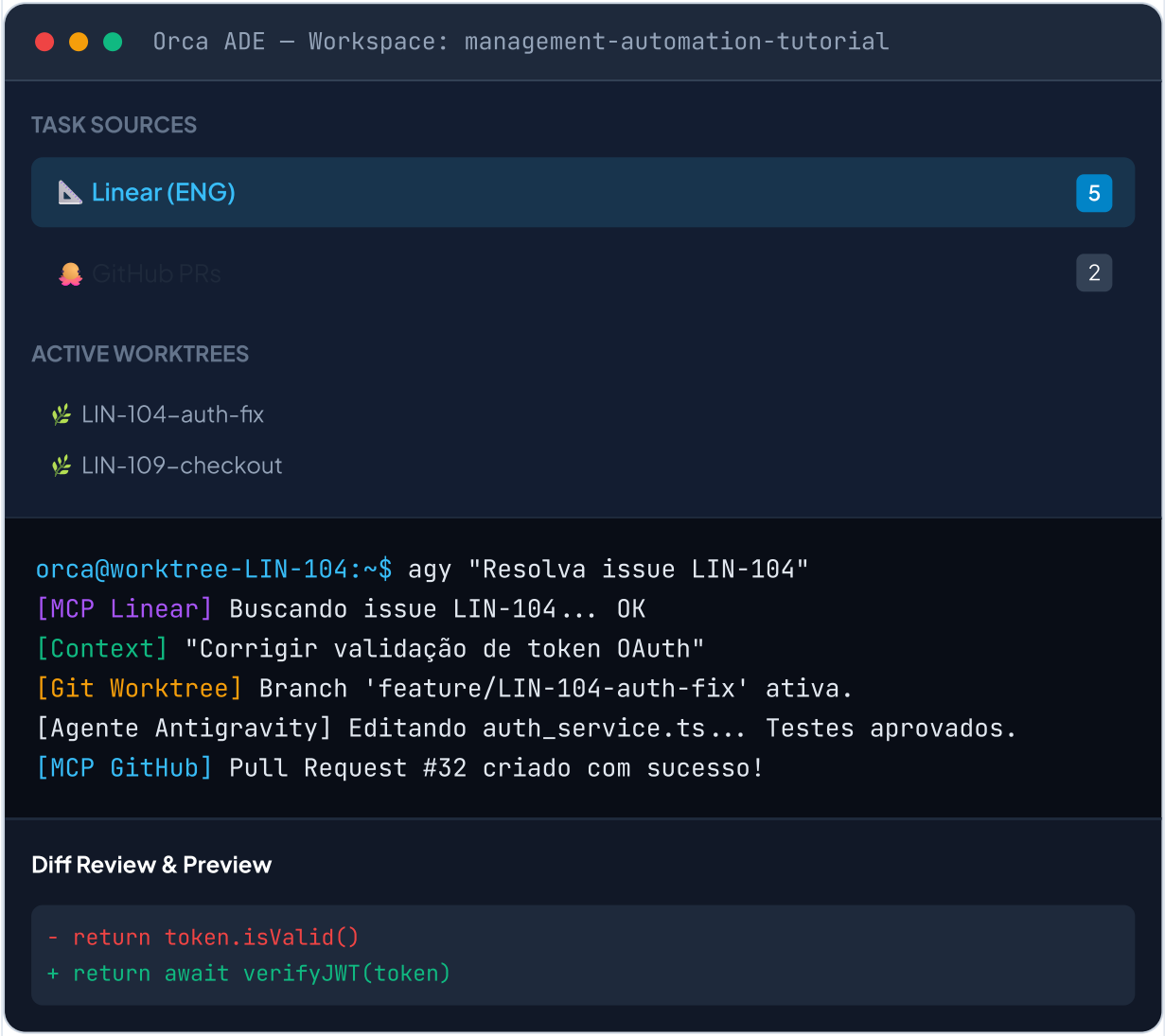
Necessário para leitura de repositórios, criação de branches e abertura de Pull Requests pelo MCP.

1. Acesse GitHub → Developer Settings → Personal access tokens.
2. Clique em **Generate new token (classic)**.
3. Dê o nome: `Orca-Agent-Integration`.
4. Marque os escopos essenciais:
   - `repo` (Acesso completo a repositórios privados e públicos)
   - `workflow` (Execução e verificação de GitHub Actions)
   - `read:org` (Listar organizações da equipe)
5. Copie o token gerado (formato: `ghp_...`).

MÓDULO 03

Conexão Nativa do Linear & GitHub no Orca

O Orca possui telas nativas dedicadas a visualizar os tickets do Linear e PRs do GitHub diretamente no cockpit da aplicação:



Configurando passo a passo no Orca

1. Abra o Orca e clique no ícone de engrenagem **Settings** (canto inferior esquerdo).
2. Acesse **Task Sources** → Marque **Linear** e **GitHub** como **ON**.
3. Acesse **Integrations**:
   - **Linear:** Clique em Conectar via OAuth ou cole sua `Personal API Key` gerada.
   - **GitHub:** Conecte sua conta do GitHub via OAuth ou insira o Personal Access Token.
4. Volte à tela inicial: os cards de tarefas aparecerão organizados em colunas (Todo, In Progress, Done).

MÓDULO 04

Configuração do MCP (Model Context Protocol)

> ⚠️ **Entenda a Diferença Essencial:**
>
> A integração visual das configurações do Orca é para **você (humano)** clicar e ver cards. A **camada MCP** é o protocolo que dá **ferramentas ao Agente de IA** para ele poder ler tarefas, pesquisar no Linear e abrir PRs no GitHub de forma 100% autônoma!

Crie um arquivo chamado `.mcp.json` na raiz do seu repositório de trabalho (ou configure via *Settings → Repository → MCP Configs* no Orca):

`.mcp.json (Configuração Pronta para Uso)`

```
{
  "mcpServers": {
    "linear": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-linear"
      ],
      "env": {
        "LINEAR_API_KEY": "lin_api_COLE_SEU_TOKEN_DO_LINEAR_AQUI"
      }
    },
    "github": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-github"
      ],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "ghp_COLE_SEU_TOKEN_DO_GITHUB_AQUI"
      }
    }
  }
}
```

Ferramenta MCP	Origem	O que a IA Consegue Fazer
`linear_search_issues`	Linear MCP	Busca issues por texto, status, tags ou projeto

Ferramenta MCP	Origem	O que a IA Consegue Fazer
`linear_get_issue`	Linear MCP	Lê detalhes, descrição completa e comentários do ticket
`linear_update_issue`	Linear MCP	Muda status para "In Progress", "In Review" ou "Done"
`create_pull_request`	GitHub MCP	Abre PR com título, descrição gerada e link para a issue
`search_code`	GitHub MCP	Procura funções, arquivos e dependências no repositório

MÓDULO 05

Configuração Universal de IA Generativa & Agentes no Orca

O Orca foi construído para ser **completamente agnóstico a modelos e agentes de IA**. O **Google Antigravity (`agy`)** é apenas um exemplo de motor; o mesmo fluxo funciona para qualquer ferramenta moderna de codificação autônoma (Claude Code, Cursor CLI, Aider, OpenCode ou modelos locais).



Agente / Motor	Tipo	Como Instalar no Sistema	Comando no Orca
Google Antigravity	CLI Autônomo	Binário oficial ou script de setup	`agy` ou `agy "instrução"`
Claude Code (Anthropic)	CLI Autônomo	`npm install -g @anthropic-ai/claude-code`	`claude`
Cursor CLI	Headless / IDE	Integrado ao Cursor IDE	`cursor-agent`
Aider	Pair Programming	`pip install aider-chat` ou `uv tool install aider-chat`	`aider`
OpenCode / Codex	Open Source CLI	`npm install -g opencode-ai`	`opencode`
Modelos Locais (Ollama)	Endpoint Local	`ollama run qwen2.5-coder`	`aider --model ollama/...`



1

Configurar o Executável (Settings ➔ Agents & Runners)

No Orca, vá em **Settings (engrenagem) ➔ Agents & Runners**:

- **Command / Executable:** O nome do executável (ex: `claude`, `agy`) ou seu *Caminho Absoluto*.
- **Default Arguments:** Parâmetros padrão (ex: `--dangerously-skip-permissions` ou `--yes`).
- **Environment Variables:** Adicione chaves globais (`ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `GEMINI_API_KEY`).

2

Criar a Worktree e Disparar a Execução

Clique com o botão direito na tarefa do Linear ➔ **"Create Worktree from Task"**. O Orca abre o terminal WebGL na pasta isolada com acesso instantâneo ao `.mcp.json`. Dispare o comando da IA para resolver a tarefa e abrir o PR.

🛠️

A grande maioria das falhas de execução ocorre por configurações de ambiente do Sistema Operacional. Veja como identificar e resolver cada caso:

🔴 Erro 1: "Command not found" / "Não foi possível encontrar a instalação"

Por que ocorre? Aplicativos de interface gráfica (como o Orca) não herdam o perfil completo do seu terminal (`.bashrc`, `.zshrc` ou alterações recentes de PATH no Windows). O pacote foi instalado globalmente pelo npm/pip, mas o Orca não o encontra.

```
Como diagnosticar no terminal do Orca:
Windows: where claude  |  where agy
macOS/Linux: which claude  |  which agy
```

✅ Solução Definitiva: Usar o Caminho Absoluto nas configurações do Orca

Substitua o comando simples pelo caminho completo do arquivo executável:

- **Windows (npm global):** `C:\Users\SEU_USUARIO\AppData\Roaming\npm\claude.cmd`
- **Windows (Python pip):** `C:\Users\SEU_USUARIO\AppData\Local\Programs\Python\Python313\Scripts\aider.exe`
- **macOS / Linux:** `/usr/local/bin/claude` ou `~/.nvm/versions/node/v20.x/bin/claude`

⚠️ Erro 2: "Script desabilitado / Execution Policy" (Windows)

Sintoma: O terminal exibe *"claude.ps1 não pode ser carregado porque a execução de scripts foi desabilitada neste sistema"*.

✅ Solução:

Abra o PowerShell como Administrador e execute uma única vez:

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

🔑 Erro 3: "API Key Missing" / Chave da IA não detectada

Sintoma: O agente inicia e fecha imediatamente dizendo *"No API key provided"*.

✅ Solução:

Defina a chave em **Settings ➔ Agents & Runners** no Orca, ou crie um arquivo `.env` na raiz da pasta:

```
ANTHROPIC_API_KEY=sk-ant-api03-...
OPENAI_API_KEY=sk-proj-...
GEMINI_API_KEY=AIzaSy...
```

⏳ Erro 4: Agente Congelado Aguardando Confirmação Interativa (y/n)

Sintoma: O processo não avança porque está solicitando permissão manual para rodar comandos bash em segundo plano.

✅ Solução:

Configure flags não-interativas ou permissões autônomas nas configurações de inicialização:

- **Claude Code:** `claude --dangerously-skip-permissions`
- **Aider:** `aider --yes --auto-commits`
- **Antigravity:** `agy --auto-approve`

MÓDULO 06

Superpoderes do Orca no Dia a Dia

Recursos essenciais que você e sua equipe devem dominar para extrair 10× mais produtividade:



Git Worktrees Paralelos

Execute 3 ou 5 agentes de IA em tarefas diferentes simultaneamente. Cada um trabalha em sua pasta sem sobrepor nem criar conflito com seus arquivos locais.



Terminais WebGL Persistentes

Terminais ultra-rápidos com scroll infinito. Se você fechar a janela ou reiniciar o PC, os processos continuam exatamente de onde pararam.



Design Mode (Inspetor Visual)

Clique em qualquer botão ou elemento da aplicação no navegador embutido do Orca. Ele copia HTML, CSS e screenshot direto para o prompt da IA.



Anotação de Diffs de IA

Revise o código gerado linha por linha antes do commit. Adicione comentários no diff e clique em "Reenviar ao Agente" para refinamento instantâneo.

MÓDULO 07

Metodologia Ágil no Linear: Uso Prático, Regras, PRD & Sprints

Para que a automação com agentes de IA (como o Antigravity) e o Orca funcione com máxima precisão, o **Linear não pode ser usado como um bloco de notas desorganizado**. Ele é o **cérebro de dados estruturados** que orienta desenvolvedores e agentes.

🎯

O Linear adota a cultura *keyboard-first* (ultra-rápido, sem burocracia de ferramentas pesadas como Jira). Nenhum código é alterado sem uma issue aberta. O ciclo de vida do card é sincronizado nativamente com o Orca e o GitHub:

Estado no Linear	Gatilho no Orca / Git	Responsável / Ação
Backlog / Triage	Ideias, débitos e bugs reportados	PM e Tech Lead priorizam e validam o DoR
Todo / Ready	Planejada no ciclo (Sprint) atual	Pronta para ser puxada por humano ou IA
In Progress	Orca cria o **Git Worktree** isolado	Agente Antigravity lê a issue via MCP e codifica
In Review	Pull Request aberto no GitHub	Agente atualiza o ticket via MCP e time faz Code Review
Done	PR mergeado na branch `main`	Fechamento automático pelo webhook do GitHub

📋

⚠️ Regra de Ouro para Agentes de IA:

Se uma tarefa for ambígua para uma pessoa, o Agente de IA irá **alucinar ou construir a coisa errada com altíssima velocidade**. A assertividade do código gerado pela IA é diretamente proporcional à qualidade do ticket no Linear!

❌ Títulos Proibidos (Vagos)

- Bug no login
- Arrumar front-end
- Melhorar performance do banco
- Ajustes no checkout

✅ Títulos Corretos (Taxonomia Padrão)

- [Auth/JWT] Implementar renovação via refresh token em /auth/refresh
- [Checkout] Corrigir cálculo de frete para pacotes acima de 2kg
- [DB/Migration] Criar índice na coluna user_id em audit_logs
- [UI/Cart] Desabilitar botão de compra quando estoque for zero

Template de Issue (Copie e cole no Linear)

```
### 1. Contexto & Problema
Qual o comportamento atual e por que precisa ser alterado.

### 2. Escopo Técnico & Arquitetura
- O que DEVE ser alterado (endpoints, tabelas, componentes).
- O que NÃO deve ser alterado (fronteiras de escopo para evitar refatorações indese

### 3. Critérios de Aceitação (Definition of Done)
- [ ] O endpoint responde 200 com novo token em payload válido.
- [ ] Retorna 401 caso o token esteja revogado ou expirado.
- [ ] Testes unitários cobrem cenários de sucesso e falha.

### 4. Anexos & Referências
- Link do PRD ou documento de arquitetura no Linear Documents.
- Link do Figma / Prints / Logs de erro para reprodução.
```

📖

O PRD moderno não é um documento burocrático de 50 páginas: é o **mapa de contexto estratégico e arquitetural**. Na era da IA, ele cumpre um papel crítico:

Âncora de Contexto Macro

Uma issue isolada diz *"o que fazer"*, mas o PRD explica *"por que e como se conecta ao restante do sistema"*. Impede decisões arquiteturais isoladas que quebram o app.

Consumível via MCP

O PRD vive dentro do **Linear Documents** vinculado ao Projeto. O agente Antigravity consulta o projeto via MCP e obtém as diretrizes antes de codificar.

Delimitação In/Out Scope

Define explicitamente o que NÃO faz parte desta entrega, impedindo que o modelo de IA tente reescrever arquivos alheios ou inventar funcionalidades.

O Linear abandona a rigidez de sprints pesadas e introduz os **Cycles**: cadência contínua com foco em velocidade, previsibilidade e automação:

1

Configuração do Ciclo (Cadência de 2 Semanas)

Vá em **Team Settings** → **Cycles** no Linear. Ative ciclos de **2 semanas** com início às segundas-feiras. Ative o `Auto-rollover` (tarefas não finalizadas passam para o próximo ciclo com alerta) e `Auto-archive` para tarefas concluídas.

2

Fatiamento Atômico de Issues (Grooming)

Durante o refinamento, o Tech Lead fatiará o PRD em **Issues Atômicas**.
Regra de Ouro: Nenhuma issue deve demandar mais de 1 ou 2 dias de trabalho. Se a estimativa for ≥ 8 pontos, a tarefa DEVE ser dividida em sub-issues menores.

3

Planning com Velocity e Escopo Congelado

O Linear calcula automaticamente a capacidade histórica do time (*Historical Velocity*). Na reunião de Planning, selecione do Backlog apenas o montante que cabe na capacidade.
Regra do Escopo Congelado: Não se adicionam novos cards no meio do ciclo sem retirar outro de mesmo peso (exceto bugs críticos P1/Urgent).

4

Acompanhamento com Burndown Chart e Scope Graph

Acompanhe diariamente pelo painel nativo do Cycle:

- **Burndown Chart:** Mostra se o ritmo de queima de pontos está alinhado com a data final.
- **Scope Graph:** Alerta visualmente se o escopo aumentou no meio da sprint (*Scope Creep*).

MÓDULO 08

Solução de Problemas (Troubleshooting)

Sintoma	Causa Provável	Solução Rápida
Servidor MCP com status vermelho / disconnected	Node.js/npx ausente ou token inválido	Rode `node -v` no terminal. Verifique se o token no `.mcp.json` não contém espaços ou aspas extras.
"Resource not found" ao listar issues	Token sem permissão no workspace do Linear	Gere novamente o token dentro do Workspace correto em *Settings → Security & Access*.
Orca não reconhece o arquivo `.mcp.json`	Arquivo fora da raiz do projeto	Certifique-se de que `.mcp.json` está na mesma pasta raiz que você abriu no Orca.
Porta já em uso no preview local (ex: :3000)	Múltiplos worktrees rodando simultaneamente	Execute o dev server do segundo worktree em outra porta: `npm run dev -- --port 3001`.

Tutorial desenvolvido para automação ágil com Orca, Linear, GitHub e Antigravity via MCP.

Pressione `Ctrl + P` a qualquer momento para imprimir ou salvar em PDF.